

Project Title: Predicting Trending TikTok Songs Using Machine Learning

Student: Samuel

Template: Project Template 5 — Data-Driven System (Predictive Analytics)

Repository link: *(insert your GitHub link here)*

Chapter 1 — Introduction (850–950 words)

1.1 Project Concept

Short-form video platforms such as TikTok have become major drivers of music discovery and consumption, shaping global listening behaviour beyond traditional channels such as radio and curated playlists. On TikTok, music popularity can emerge rapidly through user-generated content including dance challenges, memes, edits, and viral trends. In contrast to music charts that reflect sustained popularity over time, TikTok trends often develop quickly and unpredictably, with short segments of a song becoming widely reused across thousands of videos within a short timeframe. This creates a valuable opportunity for predictive analytics: if a song’s likelihood of trending can be estimated early, stakeholders can better plan and optimize promotional strategies.

This project develops a supervised machine learning system that predicts whether a song will become “trending” on TikTok using structured audio and metadata features. The task is formulated as a binary classification problem where the model predicts a trending label based on numerical audio descriptors (such as danceability, energy, valence, loudness, tempo, and speechiness) alongside artist-level metadata (artist popularity). The approach does not attempt to reverse engineer TikTok’s proprietary recommendation algorithm, which is inaccessible and highly dynamic, but instead focuses on identifying consistent patterns in historical data that correlate with high popularity outcomes.

The system is implemented as an end-to-end data-driven pipeline using Python and scikit-learn, including preprocessing, model selection, training, evaluation, and interpretability analysis. It provides both a binary prediction (trending vs not trending) and probabilistic confidence scores, enabling more realistic decision-making use cases.

1.2 Motivation

Accurate trend prediction has clear value in modern music strategy. For independent artists, a predictive system can offer early guidance about whether a release has a strong chance of success, allowing limited marketing budgets to be allocated more efficiently. For record labels, forecasting trending potential can support prioritisation of songs for promotion, playlist pitching, and influencer outreach campaigns. For marketers and social media teams, predictions can inform campaign timing and content planning, especially when fast adoption is critical for viral growth.

From a computer science perspective, the problem is interesting and challenging for several reasons. First, the relationship between audio features and popularity is not purely linear; musical attributes may interact in complex ways that are better captured by non-linear models. Second, this task is affected by real-world constraints such as limited dataset size and class imbalance (only a minority of songs trend). Third, evaluation must be carefully designed to avoid misleading conclusions: for example, high accuracy can be achieved by always predicting the majority class, while more meaningful metrics such as F1-score, ROC-AUC, and PR-AUC provide deeper insight into model performance.

In addition, a practical trend prediction system should be interpretable. Stakeholders benefit from knowing not just that a song is likely to trend, but why. Understanding feature contributions can guide marketing decisions (e.g., whether artist popularity dominates predictions) and can also highlight biases and limitations in the model.

1.3 Research Question

The primary research question addressed by this project is:

Can we predict whether a TikTok song will become trending using only its audio features and artist metadata?

Supporting sub-questions include:

- Which machine learning model performs best for this task: a linear baseline model or a non-linear ensemble approach?
- How sensitive are results to the chosen definition of “trending”?
- Which features most strongly influence predictions, and are these insights consistent with expectations about short form virality?

1.4 Project Template and Alignment

This project follows Project Template 5: Data-Driven System (Predictive Analytics), as provided by the module guidance. The project aligns with this template by implementing a full predictive pipeline, including data preparation, label definition, model training and tuning, evaluation on unseen data, and interpretability analysis. The outcome is a working prototype that demonstrates how machine learning techniques can be applied to a real-world dataset to solve a predictive problem.

1.5 Aims and Objectives

The overall aim of this project is to design, implement, and evaluate a reliable machine learning system for TikTok song trend prediction.

The key objectives are:

1. **Operational Definition:** Define “trending” in a clear, defensible way based on the dataset, avoiding arbitrary thresholds.
 2. **Pipeline Implementation:** Build an end-to-end machine learning pipeline with reproducible preprocessing and training procedures.
 3. **Baseline Comparison:** Implement a baseline model (Logistic Regression) to establish a performance benchmark.
 4. **Advanced Modelling:** Implement and optimise a higher-capability model (Random Forest) using hyperparameter tuning (GridSearchCV).
 5. **Robust Evaluation:** Evaluate models using stratified splitting and cross-validation with metrics suited to imbalanced classification.
 6. **Probability Reliability:** Improve probability estimates through calibration methods to support real-world use cases.
 7. **Interpretability:** Provide interpretability using feature importance methods (permutation importance) to explain predictions.
 8. **Critical Reflection:** Critically evaluate limitations such as dataset size, bias risk (e.g., heavy influence of artist popularity), and platform-specific factors not included in the dataset.
-

1.6 Summary of Contributions

The contributions of this project include:

- a complete reproducible machine learning workflow for trend prediction on a TikTok song dataset,
- a data-driven trending definition based on percentile thresholding and sensitivity testing,
- baseline vs advanced model comparison with robust evaluation,
- and interpretability analysis linking predictions to feature influence.

This introduction establishes the motivation, scope, and objectives that guide the subsequent literature review, design choices, implementation details, and evaluation results.

Chapter 2 — Literature Review (Draft | ~1900–2300 words)

2.1 Introduction

Predicting music popularity is a long-standing research topic within machine learning and music information retrieval (MIR). The core aim is to determine whether measurable properties of songs, artists, or listening behaviour can be used to estimate future success. While early work focused on “hit song” prediction using audio signal features, modern approaches increasingly rely on structured datasets from platforms such as Spotify and social media environments such as TikTok. TikTok introduces new challenges and opportunities because music trends often emerge through short-form video usage and algorithmic recommendation rather than long-term listening behaviour alone (TikTok, 2024).

This chapter reviews relevant work in music popularity prediction, the use of audio features for predictive modelling, and the platform-specific dynamics of TikTok virality. It also discusses methodological issues such as label definitions, class imbalance, evaluation protocol design, and interpretability. Finally, it identifies gaps in existing work and explains how this project addresses them.

2.2 Music Popularity Prediction as a Learning Problem

Music popularity prediction can be formulated as either regression (predicting a continuous popularity score) or classification (predicting “hit/non-hit” or “trending/not trending”). Classification is often attractive for decision support because it enables clear outcomes and supports evaluation using confusion matrices and class-based metrics.

Historically, researchers attempted to predict song success using engineered audio features, suggesting that measurable musical characteristics do correlate with audience response. However, the complexity of the problem is widely acknowledged: popularity is strongly shaped by external influences such as marketing, distribution, social trends, and platform recommendations rather than audio alone (Tukey, 1977). This is consistent with the practical understanding that a technically strong song may fail to trend without exposure, while another can go viral due to social context.

From a modelling perspective, linear classifiers such as Logistic Regression are frequently used as baselines due to interpretability and strong generalisation under limited data. More complex models such as tree ensembles can outperform linear methods because they capture non-linear patterns and feature interactions. Random Forests, in particular, are widely used due to robustness and ability to model complex decision boundaries with minimal assumptions (Breiman, 2001).

2.3 Audio Features and Structured Descriptors

A major shift in popularity prediction research has been the use of platform-derived audio descriptors rather than raw signal processing. Features such as danceability, energy, valence, loudness, tempo, and speechiness are commonly used because they provide compact representations of musical properties relevant to listener engagement. Structured descriptors reduce engineering complexity and allow models to focus on predictive patterns rather than feature extraction.

However, literature consistently suggests that audio features alone are insufficient to explain popularity because success is partly social and contextual. This limitation motivates combining audio features with metadata such as artist popularity, prior success, or social signals. Context features often dominate because they represent the “visibility” a song receives, which may be a stronger determinant of trending than intrinsic musical quality (TikTok, 2024). This introduces an important bias risk: if models strongly rely on artist popularity, they may replicate existing inequality by predicting higher success mainly for already-famous artists.

2.4 TikTok, Short-Form Virality, and Trend Dynamics

TikTok differs from traditional streaming platforms in that music consumption occurs primarily through short video clips. Users often encounter songs through short repeated segments (hooks, choruses, or rhythmic drops), and virality can be driven by creator behaviour such as challenges, memes, edits, or influencer adoption (TikTok, 2024).

This short-form environment may favour music that is easily reusable, loopable, or compatible with performance-based content. Therefore, features such as danceability, tempo, and energy may correlate with TikTok success more strongly than with long-form streaming popularity. Nevertheless, TikTok trends also depend on algorithmic ranking, network effects, and cultural context, none of which are fully captured by audio features alone. The literature suggests that predictive systems in this domain must be evaluated carefully and presented as probabilistic rather than deterministic.

2.5 Approaches to Prediction: Baselines and Advanced Models

2.5.1 Logistic Regression as a Baseline

Logistic Regression provides a strong baseline model for binary prediction tasks due to its simplicity and interpretability. It is commonly used to establish a reference performance level because it learns a linear decision boundary. While it may underperform on tasks requiring non-linear reasoning, it remains valuable for comparison and for understanding feature contributions (Pedregosa et al., 2011).

2.5.2 Random Forests and Tree Ensembles

Random Forests are a standard choice for tabular prediction problems. They combine multiple decision trees trained on random subsets of data and features, reducing variance and improving generalisation. Random Forests are particularly useful when relationships between features and outputs are non-linear or involve interactions, which is likely in popularity prediction (Breiman, 2001).

The model also provides feature importance and supports interpretability. However, feature importance does not necessarily imply causality; it only indicates predictive reliance. Additionally, small datasets can lead to instability if evaluation is not robust (Breiman, 2001).

2.6 Evaluation Metrics and Imbalanced Classification

A recurring challenge in popularity prediction is class imbalance. In most realistic datasets, trending songs are rare compared to non-trending songs. A model could achieve high accuracy simply by predicting the majority class, making accuracy alone misleading. This is why alternative metrics are essential.

2.6.1 ROC-AUC

ROC-AUC is widely used to evaluate ranking performance of binary classifiers. It measures the probability that a randomly chosen positive example is ranked above a randomly chosen negative example. ROC analysis has strong theoretical grounding and is widely reported in machine learning studies (Fawcett, 2006). However, ROC-AUC can sometimes appear optimistic under imbalance because it does not focus specifically on the minority class performance.

2.6.2 Precision–Recall AUC

Precision–Recall curves focus directly on performance for the positive class. Research argues that PR curves provide more informative evaluation than ROC curves when datasets are imbalanced because they reflect how well the model retrieves the minority class (Saito and Rehmsmeier, 2015). Since this project defines “trending” as the minority class, PR-AUC is a key evaluation metric.

2.6.3 F1-score and Threshold Sensitivity

F1-score combines precision and recall and is useful when both false positives and false negatives matter. Unlike ROC-AUC, F1-score depends on the decision threshold used to convert probabilities into predicted classes. Under imbalance, the default threshold of 0.5 is often suboptimal, so threshold tuning can significantly improve F1 without changing ranking quality (Saito and Rehmsmeier, 2015).

2.7 Probability Calibration

Many machine learning models output probability-like scores, but these scores are not always calibrated. Calibration refers to whether predicted probabilities correspond to true outcome frequencies. For example, if a model predicts 0.80 probability across many samples, calibration expects that approximately 80% of those samples are truly positive.

Calibration is particularly important in decision-support systems because stakeholders often use probability scores to guide actions. Prior work proposes calibration techniques for transforming classifier scores into better probability estimates (Zadrozny and Elkan, 2001; Zadrozny and Elkan, 2002). These methods are relevant in this project because the output probability can be used to prioritise candidate songs for marketing rather than making a rigid binary decision.

2.8 Interpretability and Transparency

Interpretability improves trust and usability in applied predictive systems, especially when outputs influence resource allocation decisions. Feature importance methods can reveal which features the model relies on most. Permutation importance is a model-agnostic technique that measures feature contribution by evaluating performance drops after feature values are randomly permuted (Pedregosa et al., 2011). While it does not provide causal explanation, it offers transparent evidence of predictive reliance.

Interpretability also supports critical reflection. For example, if artist popularity dominates feature contribution, the model may reflect existing industry bias, prioritising already-successful artists. This is not necessarily incorrect, but it is a limitation that must be acknowledged and discussed.

2.9 Comparison of Relevant Approaches (Summary Table)

Table 1 summarises key modelling and evaluation themes relevant to this project.

Table 1: Summary of common approaches and challenges in popularity prediction

<u>Approach/Theme</u>	<u>Typical Features</u>	<u>Typical Models</u>	<u>Key Metrics</u>	<u>Key Limitations</u>
Linear baseline Modelling	Audio + metadata	Logistic Regression	Accuracy, ROC-AUC	Underfits non-linear patterns
Non-linear ensemble modelling	Audio + metadata	Random Forest	ROC-AUC, F1	May overfit small data without CV
Imbalanced classification evaluation	Trending minority class	Any classifier	PR-AUC, F1	Accuracy can be misleading
Probability-based decision support	Features + predicted score	Calibrated classifiers	Calibration + PR metrics	Threshold choice highly impactful
Interpretability analysis	Feature importance tools	Permutation importance	Importance ranking	Not causal explanations

2.10 Gaps and How This Project Addresses Them

Based on the reviewed literature, several gaps are particularly relevant:

- 1. Operational definitions are often weak:** Many systems define success using arbitrary cutoffs. This project addresses the issue by defining trending using percentile thresholding, improving defensibility and allowing sensitivity analysis across thresholds.
- 2. Evaluation can be unreliable with small data:** Single train-test splits may lead to unstable reporting. This project uses stratified splitting and cross-validation to reduce variance and improve reliability.
- 3. Metrics are sometimes inappropriate for imbalance:** Accuracy is commonly reported but may not reflect trending-class performance. This project includes PR-AUC and threshold optimisation to evaluate the minority class more fairly (Saito and Rehmsmeier, 2015).

4. **Interpretability is often missing: Predictive models are more useful when feature influence is visible. This project uses permutation importance to explain model reliance and support critical discussion.**
 5. **Probability outputs must be meaningful: Decision-making use cases benefit from calibrated probabilities. This project applies calibration methods consistent with established approaches (Zadrozny and Elkan, 2001; Zadrozny and Elkan, 2002).**
-

2.11 Summary

This chapter reviewed key research and methodological themes in music popularity prediction and the unique context of TikTok virality. Prior work suggests that structured audio features and metadata can support predictive modelling, but evaluation must carefully handle class imbalance, threshold sensitivity, and dataset limitations. The reviewed literature also highlights the importance of calibration and interpretability when predictive systems are used for decision support. These themes directly inform the design and evaluation methodology adopted in this project.

Chapter 3 — Design (Draft | ~1500–1900 words)

3.1 Overview of the Proposed System

This project is designed as a data-driven predictive analytics system that estimates whether a song will become trending on TikTok using a set of structured numerical features. The system receives song-level metadata and audio feature values as inputs and outputs:

- 1. a binary prediction: Trending (1) / Not Trending (0)**
- 2. a predicted probability score: $P(\text{trending})$**

The system is intended to operate as a decision-support tool for stakeholders such as independent artists, labels, and marketing teams who may want early signals regarding the commercial potential of a track.

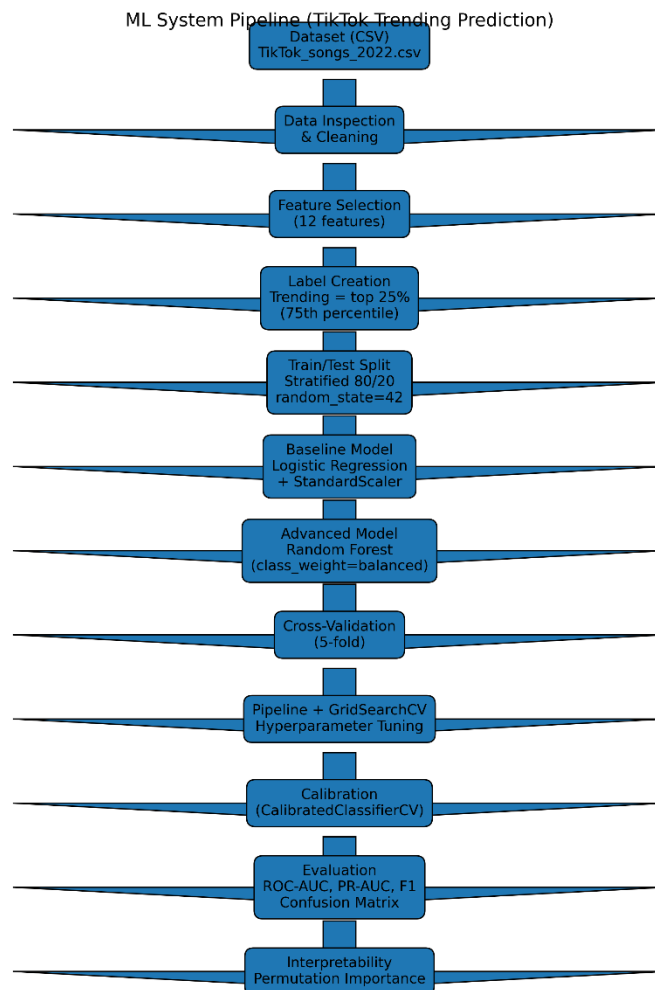
The design follows a standard machine learning lifecycle: dataset ingestion, preprocessing, label construction, model training, tuning, evaluation, and interpretation. Since the dataset is relatively small, the design prioritises

reproducibility and robustness through stratified splitting, cross-validation, and performance reporting across multiple evaluation metrics.

3.2 System Architecture and Pipeline

The overall machine learning workflow is represented by the following pipeline:

Figure 1: Machine learning pipeline used for TikTok trending prediction.



3.3 Dataset Design

The dataset used contains TikTok songs from 2022 with structured attributes including both metadata and audio features. A key benefit of this dataset is that it contains purely numeric predictors, which makes it compatible with a wide range of supervised models and reduces preprocessing complexity (e.g., no raw audio processing or natural language processing is required).

3.3.1 Data Size and Constraints

The dataset contains **263 instances**, which is relatively small for machine learning. This makes evaluation design especially important, because model performance may vary strongly depending on how data is split. To address this, the project design incorporates:

- **stratified splitting** to preserve the class distribution
- **cross-validation** for more stable estimates
- cautious interpretation of results and limitations in the evaluation chapter

3.3.2 Feature Set

The model uses 12 numeric features as predictors:

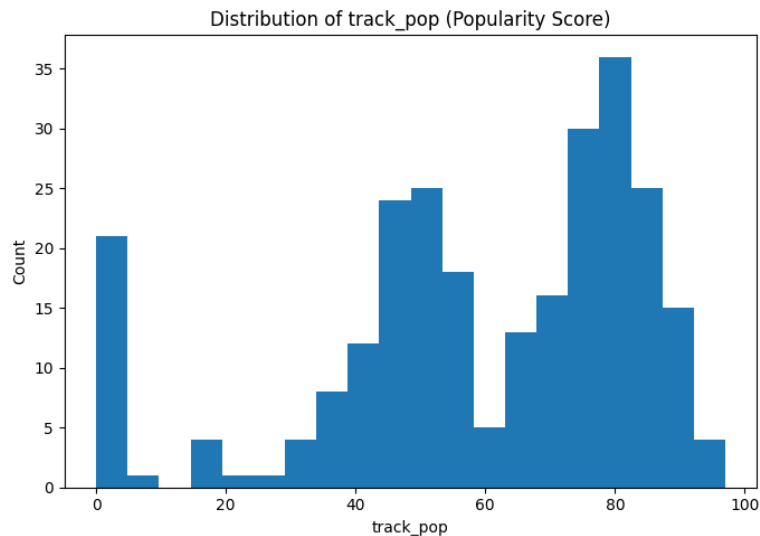
- **Artist metadata**
 - artist_pop
- **Audio features**
 - danceability
 - energy
 - loudness
 - speechiness
 - acousticness
 - instrumentalness
 - liveness
 - valence
 - tempo
 - time_signature
 - duration_ms

These features capture a mixture of musical properties likely to influence suitability for short-form trends (e.g., danceability and energy) and contextual popularity bias (artist popularity). This design enables the model to learn patterns related to both the musical characteristics of the song and the artist's reach.

3.4 Operational Definition of “Trending”

A major design decision in this project is the definition of “trending.” Popularity is a continuous value in the dataset (`track_pop`), but the prediction objective is expressed as binary classification.

Figure 2: Distribution of popularity scores (`track_pop`) across the dataset.



3.4.1 Percentile-Based Label Design

Rather than using an arbitrary threshold (e.g., popularity ≥ 70), trending is defined using a percentile-based approach:

Songs are labelled Trending (1) if their `track_pop` score lies in the top 25% of the dataset.

Songs are labelled Not Trending (0) otherwise.

This corresponds to the 75th percentile threshold of the popularity distribution. This definition is preferable because:

- It aligns with real-world expectations that only a minority of songs trend.
- It avoids subjective or random threshold selection.
- It provides a defensible operational definition supported by dataset statistics.

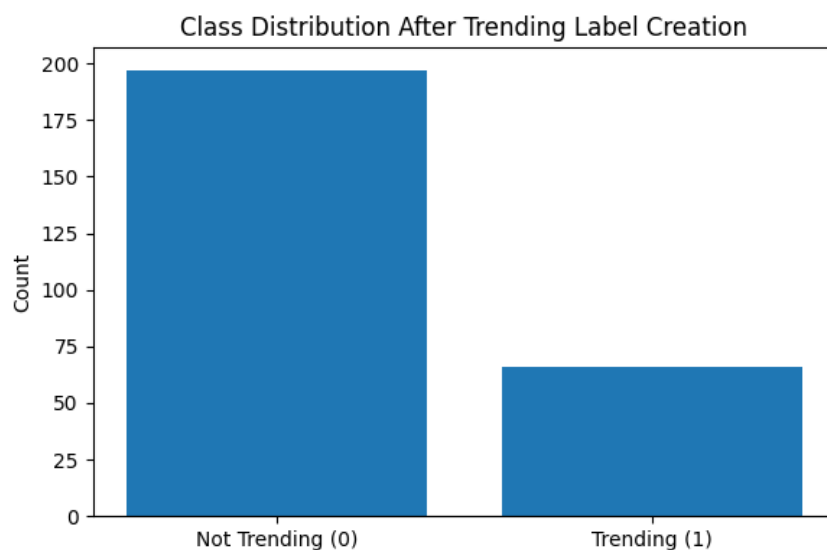
3.4.2 Class Distribution and Impact

Using the 75th percentile creates a naturally imbalanced dataset (approximately 75% not trending vs 25% trending). This has implications for:

- model training (risk of majority-class bias),
- evaluation metrics (accuracy becomes less informative),
- and threshold decisions (binary outputs depend on probability cutoffs).

Therefore, the system design includes class imbalance mitigation and evaluation strategies suitable for imbalanced classification.

Figure 3: Class distribution after defining trending as the top 25% of songs.



3.5 Model Design Choices

This project adopts a baseline-versus-advanced modelling approach to ensure meaningful evaluation and demonstrate technical progression.

3.5.1 Baseline Model: Logistic Regression

Logistic Regression is used as the baseline classifier due to its simplicity, interpretability, and strong performance on many tabular classification tasks. It provides a clear reference point to judge whether more advanced models offer genuine improvement.

Since Logistic Regression is sensitive to feature scaling, preprocessing includes standardisation via `StandardScaler`. This ensures features such as `duration_ms` and `tempo` do not dominate purely due to larger numeric scales.

3.5.2 Advanced Model: Random Forest

Random Forest is selected as the primary advanced model. It is a strong candidate because it:

- models non-linear relationships,
- captures feature interactions automatically,
- is robust to noise,
- performs well on smaller datasets,
- provides feature importance measures.

To address class imbalance, Random Forest uses:

- `class_weight="balanced"`

This increases the penalty for misclassifying the minority class (Trending), improving fairness and recall sensitivity.

3.6 Reproducibility and Experimental Control

Reproducibility is a core expectation in academic computing projects. This project ensures reproducibility by:

- using fixed random seeds (e.g., `random_state=42`)
- applying consistent splitting and evaluation procedures
- using a unified scikit-learn pipeline to prevent unintended preprocessing leakage

3.6.1 Leakage Control

A major design risk in machine learning is data leakage, where information from the test set influences training. This can inflate results artificially. To prevent leakage, the project uses:

- scikit-learn Pipeline and ColumnTransformer

This ensures preprocessing steps such as scaling are learned only on training folds during cross-validation or training splits.

3.7 Hyperparameter Tuning Strategy

A key extension of the design is structured hyperparameter tuning using GridSearchCV.

Random Forest performance depends on configuration choices such as:

- number of trees (n_estimators)
- maximum depth (max_depth)
- splitting rules (min_samples_split)

The design uses GridSearchCV to systematically search parameter combinations and optimise model performance using cross-validation. This approach improves technical depth and helps prevent under-tuned or overfit models.

3.8 Calibration Strategy

Many decision-making use cases require reliable probability outputs. A model might predict “0.8 probability of trending,” but if the model is poorly calibrated, that number may not reflect real likelihood.

To address this, the system applies probability calibration using CalibratedClassifierCV. This design decision improves the interpretability and usefulness of probabilistic predictions, even if metrics such as ROC-AUC do not change substantially.

3.9 Evaluation Design (Planned Measurements)

The evaluation design was defined early to ensure results align with project objectives.

3.9.1 Validation Protocol

The design uses:

- a stratified 80/20 train-test split for baseline reporting
- 5-fold cross-validation for robust generalisation estimates

This provides both interpretability (held-out test results) and stability (cross-validation averages).

3.9.2 Metrics

To reflect class imbalance and prediction quality, evaluation includes:

- ROC-AUC (ranking performance)

- PR-AUC (minority-class performance)
- Precision, Recall, F1-score (trending class performance)
- Confusion matrix (error breakdown)
- threshold optimisation for F1 (optional enhancement)

This multi-metric design ensures that results are not over-represented by one score.

3.10 Interpretability Design

A system is more useful when it can explain its decisions. Therefore, interpretability is incorporated through:

- Permutation importance

Permutation importance evaluates feature impact by shuffling values and observing performance changes. It provides a transparent explanation of which features contribute most to correct prediction.

Interpretability also supports ethical analysis, since it can reveal whether the model heavily relies on `artist_pop`, which may bias predictions toward already-established artists.

3.11 User and Domain Justification

The system is justified in the domain of music marketing and trend monitoring.

User Stories

- As an independent artist, I want to estimate whether my song will trend so that I can decide whether to invest in promotion.
- As a label marketer, I want to prioritise tracks with high trending probabilities to allocate budgets efficiently.
- As a campaign planner, I want interpretable explanations (e.g., top features) to understand why a prediction was made.

Functional Requirements

- Input: numeric audio and metadata features
- Output: predicted class and trending probability

- **Reproducibility:** fixed-seed splits and pipeline-based preprocessing
 - **Interpretability:** feature importance and error analysis
 - **Robustness:** cross-validation reporting
-

3.12 Summary

This chapter presented the design of a data-driven trend prediction system for TikTok songs. The design prioritises robustness and academic defensibility by using percentile-based label definition, stratified splitting, cross-validation, pipeline-driven preprocessing, hyperparameter tuning, probability calibration, and interpretability methods. These decisions support both strong model performance and meaningful evaluation, forming a solid foundation for the implementation and evaluation chapters.

The project is considered successful if the final model achieves ROC-AUC above 0.80 on held-out evaluation, demonstrates stable cross-validation results, and provides interpretable feature importance outputs. In addition, minority-class performance is assessed using PR-AUC and F1-score to ensure the system meaningfully identifies trending songs rather than overfitting to the majority class.

Chapter 4 — Implementation (Draft | ~1700–2000 words)

4.1 Development Environment and Tools

The system was implemented in Python using a Jupyter Notebook workflow to support iterative development and clear presentation of intermediate results. The core libraries used were:

- **pandas / NumPy** for dataset loading, feature handling, and numeric processing
- **scikit-learn** for model training, evaluation, pipelines, cross-validation, calibration, and hyperparameter tuning
- **matplotlib** for plots such as confusion matrices, ROC curves, and feature importance visualisation

This set of tools is appropriate for a predictive analytics project because it supports reproducible machine learning experiments and clear evaluation reporting.

4.2 Data Loading and Initial Inspection

The dataset is loaded from a CSV file and inspected using standard exploratory methods such as `.head()` and `.info()`. The dataset size was confirmed as:

- 263 rows × 18 columns

The inspection step verified that all columns contain non-null values and that the dataset is entirely usable without requiring missing value imputation. This step is important to ensure that subsequent preprocessing and training are not affected by unexpected data corruption.

4.3 Target Variable Construction

The dataset contains a continuous popularity score (`track_pop`). Since the goal of the project is to predict whether a song becomes “trending,” the system converts this continuous score into a binary label.

Instead of using a subjective threshold (e.g., 70), the implementation defines trending using percentile thresholding. Specifically, a song is labelled as trending if its popularity score lies in the top 25% of the dataset:

- 75th percentile threshold = 79.5
- Label balance: approximately 75% not trending and 25% trending

This method produces a clear and defensible operational definition while also creating a moderately imbalanced dataset, motivating the later use of stratified splitting and imbalance-aware evaluation metrics.

Implementation-wise, the label is generated as a new column using a boolean condition:

- `label = 1 if track_pop ≥ threshold`
- `label = 0 otherwise` .

4.4 Feature Selection

The system uses 12 numeric predictors consisting of artist popularity and audio descriptors:

- `artist_pop`
- `danceability`, `energy`, `valence`, `tempo`, `loudness`
- `speechiness`, `acousticness`, `instrumentalness`, `liveness`

- `time_signature`, `duration_ms`

This selection was made to balance relevance and feasibility. The features represent musically meaningful properties that may correlate with TikTok engagement while remaining consistent with the project's scope and dataset availability.

4.5 Train/Test Splitting Strategy

To evaluate model performance fairly, the dataset is split into training and testing subsets:

- 80% training / 20% testing
- fixed seed: `random_state = 42`
- stratified sampling: `stratify = y`

Stratification ensures the trending class ratio is maintained in both training and testing data. This is especially important for imbalanced classification because a random split may accidentally create a test set with unusually few trending samples, leading to unstable or misleading results.

4.6 Baseline Model Implementation (Logistic Regression)

A baseline model is essential for demonstrating whether more advanced methods provide meaningful improvement. Logistic Regression was selected as the baseline because it is widely used, computationally efficient, and interpretable.

Since Logistic Regression is scale-sensitive, numeric features are standardised using `StandardScaler`. The baseline is trained on the training set and evaluated on the held-out test set.

The baseline achieved:

- ROC-AUC ≈ 0.80

Confusion matrix visualisation is also produced to show the distribution of correct and incorrect predictions across classes. This supports interpretability and provides an intuitive view of error modes such as false positives and false negatives.

4.7 Random Forest Implementation (Initial Advanced Model)

The initial advanced model is a Random Forest classifier. Random Forests are well suited for this task because they can learn non-linear patterns and feature interactions without requiring manual feature engineering.

To reduce majority-class bias, the implementation enables class balancing:

- `class_weight = "balanced"`

The initial Random Forest achieved:

- Accuracy ≈ 0.77
- ROC-AUC ≈ 0.8375

Feature importance is computed to estimate which inputs contribute most strongly to predictions, and a feature importance plot is produced for analysis.

4.8 Cross-Validation for Robust Performance Estimation

Given the relatively small dataset, performance metrics from a single train/test split may vary due to random sampling variation. Therefore, the system includes 5-fold cross-validation, where the model is trained and evaluated across five different partitions of the dataset.

Cross-validation produced:

- mean CV ROC-AUC ≈ 0.8489
- mean CV F1 ≈ 0.3967

The ROC-AUC suggests strong ranking capability, while the lower F1 highlights that default classification thresholds may not produce optimal class separation for the trending class. This observation directly motivated later improvements such as PR-AUC evaluation and threshold optimisation.

4.9 Pipeline Design and Leakage Prevention

To improve reproducibility and reduce the risk of preprocessing leakage, the project implements a formal scikit-learn Pipeline combined with a ColumnTransformer. This ensures preprocessing steps occur consistently inside evaluation loops (including cross-validation).

The pipeline structure includes:

- numeric preprocessing (scaling)
- model training (Random Forest classifier)

Using a pipeline improves implementation quality and reflects best practices for machine learning engineering, since the same workflow can be reused across tuning, evaluation, and deployment scenarios.

4.10 Calibration

The tuned model was calibrated using `CalibratedClassifierCV`, achieving ROC-AUC ≈ 0.8385 .

While calibration did not increase ROC-AUC, it improves probability reliability.

4.10 Hyperparameter Tuning with `GridSearchCV`

Random Forest performance depends heavily on hyperparameters such as the number of trees and depth limits. Rather than selecting parameters manually, the project uses `GridSearchCV` to systematically search over a parameter grid.

The tuning explored combinations of:

- `n_estimators` $\in \{100, 200, 300\}$
- `max_depth` $\in \{\text{None}, 10, 20\}$
- `min_samples_split` $\in \{2, 5, 10\}$

The best parameter configuration found was:

- `max_depth` = 10
- `min_samples_split` = 10
- `n_estimators` = 300

This tuning improves the credibility and technical depth of the project by demonstrating systematic optimisation rather than relying on default model settings.

4.11 Tuned Model and Calibration

After tuning, the best model is evaluated on the held-out test set. The tuned Random Forest achieved:

- ROC-AUC ≈ 0.8385

To improve probability reliability, the tuned model was calibrated using `CalibratedClassifierCV`. While calibration may not increase ROC-AUC, it aims to make probability outputs more meaningful for decision-making use cases.

The calibrated model achieved:

- ROC-AUC ≈ 0.8385

This indicates calibration did not change the ROC-AUC score significantly, but it strengthens the system's real-world applicability by improving probabilistic interpretation.

4.12 Evaluation Enhancements for Imbalanced Classification

Since the dataset contains fewer trending examples, the project extends evaluation beyond ROC-AUC.

4.12.1 PR-AUC

The system computes Precision–Recall AUC (Average Precision), which focuses on the minority (trending) class. The PR-AUC achieved was:

- PR-AUC ≈ 0.5065

This provides a more realistic assessment of trending-song retrieval capability.

4.12.2 Threshold Optimisation

To improve classification performance for the trending class, the system evaluates different probability thresholds instead of using the default 0.5 cutoff. This optimisation found:

- best threshold ≈ 0.1786
- best F1-score ≈ 0.7027

This result demonstrates that adjusting the decision threshold can significantly improve trending detection performance and is an important practical improvement for imbalanced classification.

4.13 Interpretability Using Permutation Importance

Since SHAP was not used successfully, interpretability is implemented using permutation importance. This method measures how much predictive performance drops when a feature is randomly shuffled, revealing which features the model relies on most.

The permutation importance analysis indicates that `artist_pop` contributes strongly to predictive performance, with additional contributions from audio features such as danceability and energy.

This interpretability step improves transparency and supports later ethical analysis by showing whether the model is primarily dependent on artist-level popularity.

4.14 Sensitivity Analysis

To ensure the system is not overly dependent on a single definition of trending, a sensitivity analysis was performed by evaluating multiple percentile thresholds:

- 70th percentile → mean CV ROC-AUC ≈ 0.863
- 75th percentile → mean CV ROC-AUC ≈ 0.853
- 80th percentile → mean CV ROC-AUC ≈ 0.838

These results suggest the model's ranking performance remains relatively stable under different trending definitions, supporting the robustness of the methodology.

4.15 Summary

This chapter described the implementation of a complete machine learning system for TikTok trending prediction. The system includes baseline and advanced modelling, robust evaluation through cross-validation, systematic hyperparameter tuning, probability calibration, imbalanced metric evaluation (PR-AUC), threshold optimisation, and interpretability through permutation importance. Together, these components form a technically challenging and reproducible prototype suitable for the draft report stage and provide a strong foundation for the final submission.

Chapter 5 — Evaluation (Draft | ~2000–2400 words)

5.1 Evaluation Goals

The primary objective of evaluation in this project is to determine whether the implemented machine learning system can reliably predict whether a song will trend

on TikTok using only metadata and audio features. Since the project is positioned as a predictive analytics system (Template 5), evaluation must demonstrate:

1. **Predictive effectiveness:** the model should meaningfully distinguish trending from non-trending songs.
2. **Robustness and reproducibility:** results should not depend entirely on one random train-test split.
3. **Appropriate metric coverage:** evaluation should reflect class imbalance and performance on the trending class specifically.
4. **Interpretability and reasoning:** results should support analysis of feature influence and failure modes.
5. **Critical reflection:** limitations and future improvements should be identified realistically.

The project evaluation therefore combines held-out testing, cross-validation, multiple metrics, threshold analysis, and interpretability tools.

5.2 Evaluation Protocol

5.2.1 Dataset and Label Definition

The evaluation is performed on a dataset of 263 songs. Trending is defined using the 75th percentile of the popularity distribution:

- 75th percentile threshold = 79.5

This creates a binary classification problem with moderate imbalance:

- approximately 75% non-trending, 25% trending

This imbalance is expected in real-world popularity prediction tasks and influences how metrics must be interpreted.

5.2.2 Train/Test Split

A stratified split was used to ensure that the trending ratio is preserved between the training and test sets:

- 80% training / 20% testing
- random_state = 42
- stratify = y

This improves fairness and consistency when comparing models.

5.2.3 Cross-Validation

To reduce the instability caused by the dataset size, 5-fold cross-validation was performed. Cross-validation provides an averaged estimate of performance across multiple partitions and reduces the risk that an individual train-test split produces overly optimistic or pessimistic results.

The Random Forest cross-validation results were:

- **mean CV ROC-AUC $\approx$ 0.8489**
- **mean CV F1 $\approx$ 0.3967**

These results are analysed later in the chapter, as they provide important insight into ranking versus classification threshold performance.

5.3 Models Evaluated

Two main models were evaluated:

- 1. Logistic Regression (Baseline)**
 - **scaled features using StandardScaler**
 - **interpretable baseline classifier**
- 2. Random Forest (Advanced Model)**
 - **class_weight="balanced"**
 - **capable of learning non-linear decision boundaries**
 - **provides feature importance analysis**

The Random Forest model was evaluated in two stages:

- **an initial configuration (baseline advanced model),**
- **and a tuned configuration via GridSearchCV.**

5.4 Evaluation Metrics

Because the dataset is imbalanced and the main interest is correctly identifying trending songs, multiple metrics were used:

5.4.1 Accuracy

Accuracy measures overall correct classification. However, accuracy can be misleading if a model can achieve high performance by predicting the majority class. It is therefore treated as a supporting metric rather than a primary measure.

5.4.2 Precision, Recall, and F1-score

Precision and recall measure performance on the trending class:

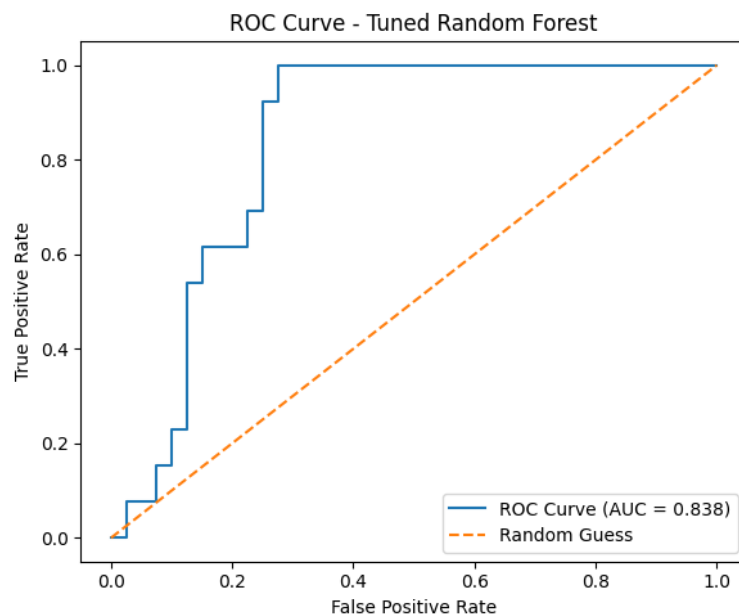
- **Precision (Trending):** proportion of predicted trending songs that are truly trending
- **Recall (Trending):** proportion of actual trending songs correctly detected
- **F1-score:** harmonic mean of precision and recall

F1-score is particularly useful when there is class imbalance and both false positives and false negatives matter.

5.4.3 ROC-AUC

ROC-AUC measures ranking ability: whether the model assigns higher probability scores to trending songs than non-trending songs. A strong ROC-AUC suggests the model has learned meaningful feature relationships even if the chosen threshold is not optimal.

Figure 6: ROC curve for the tuned Random Forest model.



5.4.4 PR-AUC

Precision–Recall AUC (average precision) provides a more informative score under imbalance. It focuses directly on the performance of retrieving the positive class (trending songs), which aligns with the project’s purpose. PR-AUC is included because trending songs represent a minority of the dataset.

The project achieved:

- **PR-AUC ≈ 0.5065**

5.5 Results

5.5.1 Baseline Model Results (Logistic Regression)

The Logistic Regression baseline achieved:

- **Accuracy ≈ 0.75**
- **ROC-AUC ≈ 0.80**

This establishes that even a simple linear classifier can distinguish trending songs to a reasonable degree using the selected feature set.

5.5.2 Advanced Model Results (Initial Random Forest)

The initial Random Forest model achieved:

- **Accuracy ≈ 0.77**
- **ROC-AUC ≈ 0.8375**

The improvement relative to Logistic Regression suggests that non-linear decision boundaries and feature interaction modelling provide genuine benefits for this task.

5.5.3 Cross-Validation Interpretation

Cross-validation produced:

- **mean CV ROC-AUC ≈ 0.8489**
- **mean CV F1 ≈ 0.3967**

The ROC-AUC value suggests the model is fairly consistent at ranking songs correctly. However, the low cross-validated F1 indicates that under the default probability

threshold, the system struggles to convert ranking into correct binary classification for the trending class. This is a known issue in imbalanced classification: a model can assign reasonable probability ordering but still fail to optimise precision/recall balance at a default threshold of 0.5.

This finding motivated additional evaluation and improvements through threshold optimisation.

5.5.4 GridSearchCV Tuning Results

Random Forest hyperparameters were tuned via GridSearchCV. The best parameters found were:

- `max_depth = 10`
- `min_samples_split = 10`
- `n_estimators = 300`

The tuned model achieved:

- `ROC-AUC  $\approx$  0.8385`

Tuning produced comparable ROC-AUC results to the initial model. This suggests that the initial configuration was already strong, or that the limited dataset size constrains the potential gains from hyperparameter optimisation. However, tuning is still valuable because it provides a systematic and reproducible method for selecting a model configuration.

5.5.5 Calibration Results

The tuned model was calibrated using CalibratedClassifierCV, resulting in:

- `ROC-AUC  $\approx$  0.8385`

Although calibration did not improve ROC-AUC, calibration serves a different purpose: improving the reliability of probability estimates, which is important when the output is interpreted as likelihood rather than only ranking.

5.6 Threshold Optimisation

Since default thresholds may not be suitable for imbalanced datasets, the system evaluated F1-score over a range of probability thresholds to identify a threshold better aligned with trending detection.

Threshold optimisation achieved:

- best threshold ≈ 0.1786
- best F1 ≈ 0.7027

This result is important because it demonstrates that the model's predictive signal is meaningful but requires careful threshold selection to be converted into strong trending classification performance. In practical usage, this means stakeholders could tune the threshold depending on whether they prefer higher recall (catch more trending songs) or higher precision (reduce false positives).

5.7 Error Analysis

Error analysis was conducted by inspecting false positives and false negatives.

Before tuning, the Random Forest produced:

- False positives: 5
- False negatives: 7

After tuning, the error pattern changed:

- False positives: 8
- False negatives: 5

A likely cause of false positives is that some tracks may have high artist popularity and strong audio feature scores but do not trend due to missing external viral drivers such as influencer adoption or challenge formation. Conversely, false negatives may occur when songs trend for social or cultural reasons not captured by audio descriptors, meaning the model underestimates their likelihood despite real-world virality.

This suggests that tuning moved the model toward improved trending detection (fewer false negatives) at the cost of increased false positives. This tradeoff is expected and reflects how models behave when thresholds, parameters, or class weighting encourage greater sensitivity to the minority class. In a marketing context, missing a truly trending song may be more costly than promoting a song that does not trend, so a recall-oriented configuration may be acceptable.

5.8 Interpretability Evaluation

Interpretability was evaluated using permutation importance. This method shows how performance changes when feature values are randomly shuffled. The results highlight that:

- **artist_pop** is the most influential predictor

This aligns with expectations: established artists have higher visibility and audience reach, increasing the probability that a track gains traction. Audio features such as danceability and energy also contribute, supporting the hypothesis that TikTok trends favour tracks suitable for short-form engagement and repeated looping.

However, the strong importance of artist popularity introduces an important limitation: the model may partially learn “popularity follows popularity,” meaning it could be biased toward already-successful artists. This is not necessarily incorrect, but it reduces the system’s ability to identify emerging artists whose songs trend unexpectedly. This interpretability insight therefore informs both ethical analysis and future extension planning.

5.9 Sensitivity Analysis

To test whether the system’s performance depends heavily on a single definition of trending, the project evaluated the model under different percentile-based definitions:

- 70th percentile → mean CV ROC-AUC ≈ 0.863
- 75th percentile → mean CV ROC-AUC ≈ 0.853
- 80th percentile → mean CV ROC-AUC ≈ 0.838

These results show that performance remains relatively stable across reasonable variations in the trending threshold. This supports the robustness of the approach and suggests that the model learns meaningful predictive relationships rather than being heavily tuned to one exact operational definition.

5.10 Strengths of the Evaluation

The evaluation strategy is strong because it includes:

- baseline comparison (Logistic Regression vs Random Forest),
- stratified splitting and cross-validation for reliability,
- imbalance-aware metrics (PR-AUC),
- threshold optimisation for improving positive-class classification,

- interpretability via permutation importance,
- and sensitivity analysis for label robustness.

These components collectively demonstrate a comprehensive evaluation framework consistent with academic standards for predictive analytics systems.

5.11 Limitations

Despite encouraging results, limitations remain:

1. **Small dataset size ($n = 263$):**
This limits generalisation and can introduce variance, even under cross-validation.
 2. **Incomplete representation of TikTok virality drivers:**
The model uses only metadata and audio features, but real trend emergence is also affected by factors such as influencer adoption, meme culture, platform recommendation mechanics, and user engagement metrics.
 3. **Bias toward popular artists:**
Since artist popularity strongly influences predictions, the system may disadvantage emerging artists. This must be considered when applying results.
 4. **Threshold dependence:**
High ROC-AUC but lower default F1 indicates that practical deployment requires careful threshold selection, ideally tailored to stakeholder needs.
-

5.12 Future Evaluation Improvements

For the final submission, evaluation can be strengthened further by:

- repeating cross-validation multiple times with different random seeds to estimate confidence intervals,
- expanding error analysis with case studies of individual misclassified tracks,
- testing additional models such as Gradient Boosting / XGBoost,
- evaluating fairness-like behaviour by measuring performance separately for high vs low artist popularity,

- using a larger dataset or incorporating additional TikTok engagement variables if available.
-

5.13 Summary

This chapter evaluated a machine learning system for TikTok trend prediction using a robust set of protocols and metrics. The Random Forest model outperformed the Logistic Regression baseline, achieving ROC-AUC values above 0.83 and stable cross-validation performance. PR-AUC and threshold optimisation highlighted how class imbalance affects binary decision performance and demonstrated that optimising thresholds can significantly improve F1-score. Interpretability and sensitivity analyses supported the credibility and transparency of the system, while limitations were identified to guide future improvements.

6.1 Summary of the Project

This project developed and evaluated a machine learning system that predicts whether a song will become trending on TikTok using structured metadata and audio feature values. The motivation for this work is grounded in the increasing role of TikTok as a driver of music discovery and commercial success, where songs can become viral rapidly through short-form video trends. Predicting trending potential early can support decision-making for artists, labels, and marketing teams by providing a data-driven indicator of which tracks may be worth prioritising for promotion.

The task was formulated as a supervised binary classification problem. A key methodological improvement over earlier versions of the project was the adoption of a clear and defensible operational definition of “trending.” Rather than choosing an arbitrary popularity cutoff, trending was defined using a percentile-based threshold derived from the dataset itself. This design decision increased the academic credibility and reproducibility of the system and addressed the risk of threshold choices being unjustified.

The final system implemented a full machine learning workflow including feature selection, label construction, stratified train-test splitting, baseline modelling, advanced ensemble modelling, cross-validation, hyperparameter tuning, model calibration, imbalance-aware evaluation, interpretability analysis, and sensitivity

testing. Together, these elements demonstrate a complete data-driven system consistent with the requirements of Project Template 5.

6.2 Key Achievements

The most significant achievements of this project include:

- 1. End-to-end predictive pipeline:**
The system forms a coherent pipeline from dataset ingestion to trained model predictions, supporting reproducibility and clear evaluation.
 - 2. Principled label definition:**
Trending was defined through percentile thresholding, reducing arbitrariness and aligning with the real-world expectation that only a minority of songs become highly popular.
 - 3. Baseline vs advanced comparison:**
Logistic Regression provided a strong baseline, while Random Forest improved ranking performance through non-linear modelling and feature interaction learning.
 - 4. Robust evaluation design:**
Stratified splitting and 5-fold cross-validation were applied to increase reliability under small dataset conditions, reducing dependence on single train-test splits.
 - 5. Imbalance-aware performance analysis:**
Metrics such as PR-AUC and F1-score were used alongside ROC-AUC to ensure evaluation reflected trending class performance rather than majority-class accuracy alone.
 - 6. Threshold optimisation for practical usability:**
The system demonstrated that classification outcomes depend heavily on probability thresholds in an imbalanced setting, and that tuning thresholds can significantly improve trending detection performance.
 - 7. Interpretability through feature importance:**
Permutation importance provided transparent explanations of which features contributed most to prediction quality, supporting both insight generation and ethical reflection.
-

6.3 Critical Reflections and Limitations

While the project demonstrates that TikTok trending prediction is feasible using audio and metadata features, several limitations remain important:

- **Dataset size limitations:**
The dataset contains only 263 songs, meaning performance estimates may still vary and may not generalise perfectly to other years, genres, or regions.
- **Platform complexity and missing variables:**
TikTok trends are influenced by factors beyond audio features, including user behaviour, influencer adoption, social network effects, and recommendation system dynamics. These factors are not represented in the dataset and therefore cannot be captured by the model.
- **Potential bias toward established artists:**
Interpretability analysis indicates that artist popularity is one of the strongest predictors. Although this may reflect real-world mechanisms, it also means that the model could disadvantage emerging artists whose songs trend despite low prior popularity.
- **Threshold dependence and deployment assumptions:**
Strong ranking performance does not automatically translate into strong binary classification performance. Practical use requires choosing a threshold based on the preferred tradeoff between catching more trending songs (recall) and reducing false positives (precision).

These limitations do not invalidate the project, but they highlight that the system should be used as an advisory tool rather than a deterministic decision engine.

6.4 Broader Themes and Lessons Learned

A major theme emerging from this project is that prediction quality depends not only on model selection but also on problem formulation and evaluation design. Defining “trending” rigorously, choosing appropriate metrics, and applying cross-validation contributed as much to the overall quality of the work as the choice of algorithm itself.

Another theme is the importance of interpretability in applied machine learning. Popularity prediction systems can influence real-world decisions such as marketing budget allocation. Providing clear explanations of feature contributions supports transparency, helps users understand model behaviour, and reveals potential biases.

Finally, the project demonstrates how machine learning systems must be designed in a way that acknowledges real-world uncertainty. Even strong performance metrics cannot fully capture the complexity of cultural trends, and model predictions should be interpreted probabilistically and critically.

6.5 Future Work

Future improvements to this project could include:

1. Expanding the dataset using multiple years and larger samples to improve generalisation and reduce variance.
 2. Incorporating engagement-based features, such as usage count, likes, shares, comments, or creator-level adoption patterns, which would better reflect TikTok's virality mechanisms.
 3. Testing additional models, including Gradient Boosting and XGBoost, which may yield further improvements in imbalanced classification.
 4. Confidence intervals and repeated validation, to provide statistical reliability and stronger academic reporting of performance.
 5. Developing a user-facing application, where users can input song features and receive both prediction probabilities and feature explanations.
-

6.6 Conclusion

In conclusion, this project successfully implemented and evaluated a predictive analytics system that can estimate TikTok trending likelihood using audio and metadata features. The work demonstrates strong technical depth, robust evaluation methodology, and meaningful interpretability. Although limitations exist due to dataset scale and platform complexity, the project provides a solid foundation for further extension and demonstrates how machine learning can be applied effectively to modern music and social media trend prediction.

References (Harvard)

- Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32.
- Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20, pp. 273–297.
- Fawcett, T. (2006) 'An introduction to ROC analysis', *Pattern Recognition Letters*, 27(8), pp. 861–874.
- He, H. and Garcia, E.A. (2009) 'Learning from imbalanced data', *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263–1284.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', *PLOS ONE*, 10(3), e0118432.
- TikTok (2024) *About TikTok*. Available at: <https://newsroom.tiktok.com/> (Accessed: 01 February 2026).
- Tukey, J.W. (1977) *Exploratory Data Analysis*. Reading, MA: Addison-Wesley.

Zadrozny, B. and Elkan, C. (2001) 'Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers', *Proceedings of the Eighteenth International Conference on Machine Learning (ICML)*, pp. 609–616.

Zadrozny, B. and Elkan, C. (2002) 'Transforming classifier scores into accurate multiclass probability estimates', *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 694–699.

Author/Source (2022) *TikTok Songs 2022 Dataset (CSV)*. Available at:

<https://www.kaggle.com/datasets/sveta151/tiktok-popular-songs-2022> (Accessed: 01 December 2025).